
TP N°1: "Usabilidad y calidad de requisitos en el desarrollo de un sistema"

1. Objetivos del Trabajo Práctico

Este trabajo práctico tiene como propósito que los equipos integren y apliquen los conceptos centrales de la materia sobre especificación de requerimientos, usabilidad y gestión colaborativa de proyectos, a partir del análisis de un caso de negocio real y acotado.

Al finalizar el trabajo, cada equipo habrá logrado:

- Retomar y consolidar los conceptos previos sobre relevamiento y análisis de requerimientos.
- Definir requerimientos funcionales y no funcionales con calidad: identificación unívoca, pre y post-condiciones, precedencias y descripción clara.
- Aplicar los principios de usabilidad (en particular las 8 Reglas de Oro de Shneiderman) al análisis y diseño del módulo asignado.
- Identificar y justificar las diferencias de usabilidad entre una interfaz Mobile y una Desktop para la misma funcionalidad.
- Gestionar los requerimientos en una herramienta de trabajo colaborativo, simulando el ciclo de vida real de una tarea.
- Desarrollar wireframes de baja o media fidelidad que sean consistentes y trazables con los requerimientos especificados.
- Documentar lecciones aprendidas como práctica de gestión del conocimiento del equipo.

2. Conocimiento Previo Requerido

Para abordar este trabajo práctico, se espera que los integrantes del equipo cuenten con los siguientes conocimientos adquiridos en cursadas previas:

- Técnicas de relevamiento y análisis de requerimientos (entrevistas, casos de uso, historias de usuario).
- Conceptos básicos de ciclo de vida de desarrollo de software.
- Nociones de gestión de proyectos y división de tareas en equipo.
- Lectura comprensiva del material teórico provisto en el enunciado y el aula virtual.

3. Recursos Necesarios

Hardware	Al menos una computadora por equipo con acceso a internet.
Documentación	Enunciado del TP, material teórico del anexo y recursos del aula virtual.
Herramienta	Trello, Jira, ClickUp u otra herramienta de gestión colaborativa a elección del equipo (con justificación).
Prototipado	Figma u otra herramienta de wireframing a elección del equipo.
Caso de negocio	Documento AgendaYA – Sistema de Gestión de Agenda y Reservas (adjunto).

4. Premisas de Trabajo

- El trabajo es exclusivamente grupal. Cada grupo debe estar conformado por entre 7 y 8 integrantes. Esta conformación se mantendrá durante todos los trabajos prácticos del ciclo.
- A cada grupo se le asignará uno de los módulos del sistema AgendaYA como subproyecto. Dicho módulo deberá respetarse en todos los trabajos prácticos siguientes.
- El trabajo debe realizarse haciendo uso activo de la herramienta colaborativa seleccionada: no se admite documentación solo en papel o texto plano sin trazabilidad en la herramienta.
- Al finalizar el práctico, el equipo deberá elaborar una lista de lecciones aprendidas siguiendo las pautas del Anexo correspondiente.
- El análisis y diseño se realiza a nivel de wireframes. No se requiere ni se evaluará código.

5. Tareas a Realizar

Las tareas se presentan en el orden recomendado de ejecución. Los equipos pueden ajustar la distribución interna según su planificación, siempre que todos los puntos sean abordados.

5.1 Lectura del material teórico

Antes de comenzar el análisis, todo el equipo debe leer los materiales provistos en el aula virtual y en el Anexo del presente enunciado:

- Principios de Usabilidad y las 8 Reglas de Oro de Shneiderman.
- Evolución del Diseño y surgimiento del Diseño UX.
- PDF: Diferencias entre UX y UI (Procesos - Bere).
- Presentaciones de teoría de la cátedra.

¿Por qué es importante leer primero?

El análisis de requerimientos y el diseño de wireframes que realizarán en los puntos siguientes deben estar fundamentados en los conceptos teóricos.

Una decisión de diseño sin respaldo teórico no será evaluada como válida, aunque visualmente sea atractiva.

5.2 Análisis del caso de negocio y especificación de requerimientos

Cada equipo deberá leer el documento AgendaYA y analizar en profundidad el módulo asignado. A partir de ese análisis, deberán especificar sus requerimientos de manera formal, respetando los siguientes criterios de calidad:

Atributo requerido	Descripción y criterio de cumplimiento
Identificación unívoca	Cada requerimiento debe tener un código único definido en consenso por todo el curso en la primera clase. El código debe permitir identificar el proyecto, el tipo de requerimiento (funcional o no funcional) y el módulo al que pertenece. Ejemplo de estructura posible (no obligatoria): AYA-M04-RF01.
Tipo de requerimiento	Indicar explícitamente si es Funcional (RF) o No Funcional (RNF). El equipo debe especificar al menos 3 RF y 5 RNF por módulo asignado.

Descripción coloquial	Redacción en lenguaje natural, clara y sin ambigüedad, que permita que cualquier stakeholder (no técnico) entienda qué debe hacer el sistema.
Pre-condiciones	Estado del sistema o del usuario que debe ser verdadero antes de que el requerimiento pueda ejecutarse. Ejemplo: 'El administrador debe haber iniciado sesión.'
Post-condiciones	Estado del sistema una vez que el requerimiento se ejecutó correctamente. Ejemplo: 'El turno queda registrado en la agenda y se envía un email de confirmación.'
Precedencias	Si el requerimiento depende de que otro esté implementado o activo, debe indicarse con el código del requerimiento precedente. Si no tiene precedencias, indicar: 'Sin precedencias'.

Nota sobre la nomenclatura

En la primera clase del TP, todo el curso deberá acordar en consenso el formato de identificación de requerimientos.

El formato mostrado arriba es únicamente orientativo. El equipo NO puede utilizar el ejemplo del enunciado sin modificarlo.

Una vez acordado el nomenclador en clase, todos los grupos deben utilizarlo de manera consistente.

5.3 Selección y justificación de la herramienta de gestión

El equipo debe seleccionar una herramienta de gestión colaborativa para cargar y administrar sus requerimientos. La elección debe estar fundamentada en criterios objetivos.

Se sugieren las siguientes opciones, aunque el equipo puede proponer otras:

- Trello - tablero Kanban simple, ideal para proyectos de mediana complejidad.
- Jira - robustez y trazabilidad, orientado a entornos de desarrollo ágil.
- ClickUp - versátil, con múltiples vistas y capacidad de personalización.

La justificación debe incluir al menos los siguientes criterios de evaluación: funcionalidades disponibles en la versión gratuita, facilidad de uso para el equipo, soporte para asignación de responsables y gestión de estados, y adecuación al alcance del módulo asignado.

5.4 Carga de requerimientos en la herramienta

Una vez seleccionada la herramienta, el equipo debe cargar en ella todos los requerimientos especificados. Para cada requerimiento, la carga debe incluir como mínimo:

- Código de identificación unívoca.
- Descripción completa (incluyendo pre y post-condiciones, y precedencias).
- Responsable del requerimiento dentro del equipo.
- Estado actual dentro del flujo de trabajo definido.

La organización interna de las tareas (columnas, etiquetas, prioridades) queda a criterio del equipo, y será evaluada como parte del juicio profesional aplicado.

5.5 Simulación del ciclo de vida de los requerimientos

Para demostrar el uso real de la herramienta, el equipo debe simular el avance de los requerimientos a través de sus diferentes estados. La simulación debe mostrar al menos los siguientes estados representados en la herramienta:

- Pendiente / Backlog
- En análisis
- En desarrollo / En diseño
- En revisión
- Completado / Done

No todos los requerimientos deben estar en el mismo estado: la muestra debe ser representativa del ciclo completo, con al menos un requerimiento en cada estado relevante.

5.6 Aplicación de conceptos de UX y Usabilidad

A partir del material teórico leído y de los requerimientos definidos, el equipo debe analizar su módulo aplicando los principios de usabilidad. Se espera:

- Identificar al menos 3 de las 8 Reglas de Oro de Shneiderman que son relevantes para el módulo asignado, y justificar cómo impactan en las decisiones de diseño.
- Analizar las diferencias de usabilidad entre la versión Mobile y Desktop de la funcionalidad, cuando corresponda según el módulo.
- Documentar las decisiones de diseño tomadas en función de los principios aplicados.

Este análisis debe quedar reflejado tanto en el informe escrito como en los wireframes desarrollados en el punto siguiente.

5.7 Desarrollo de wireframes

El equipo debe diseñar los wireframes de al menos una funcionalidad completa del módulo asignado. Los wireframes deben:

- Representar el flujo completo de la funcionalidad seleccionada, desde el punto de entrada hasta la confirmación de la acción realizada.
- Ser consistentes con los requerimientos especificados: lo que se muestra en el wireframe debe corresponderse con lo que dice el requerimiento.
- Mostrar explícitamente cómo se aplican al menos 2 principios de usabilidad en el diseño de la interfaz.
- Incluir versión Mobile y versión Desktop si el módulo involucra ambos contextos de uso.

Se recomienda el uso de Figma, aunque el equipo puede optar por cualquier herramienta de prototipado que permita compartir el resultado (Balsamiq, Marvel, Penpot, Whimsical, etc.).

Nivel de fidelidad esperado

No se evalúa el acabado visual ni el estilo gráfico de los wireframes.

Lo que se evalúa es la coherencia entre el requerimiento y el flujo diseñado, la aplicación visible de principios de usabilidad, y la completitud del flujo representado.

Un wireframe de baja fidelidad (en blanco y negro, sin colores ni imágenes) es completamente válido si está bien fundamentado.

5.8 Elaboración de lecciones aprendidas

Al finalizar el trabajo práctico, el equipo debe redactar una lista de lecciones aprendidas. Este es un entregable obligatorio y será evaluado. Ver Anexo: Lecciones Aprendidas para los criterios de elaboración.

6. Entregables y Modalidad de Evaluación

6.1 Presentación en clase (Día 2)

Cada equipo realizará una presentación oral ante la cátedra que debe incluir:

- Al menos un requerimiento completo (con todos sus atributos), expuesto y defendido verbalmente.
- Los wireframes correspondientes al requerimiento presentado, demostrando su consistencia.
- Una demostración en vivo del uso de la herramienta de gestión seleccionada.
- La lista preliminar de lecciones aprendidas del equipo.

Criterio de evaluación de la presentación

No alcanza con mostrar los artefactos: el equipo debe poder explicar las decisiones tomadas.

La cátedra realizará preguntas sobre el requerimiento presentado, los principios de usabilidad aplicados y la justificación de la herramienta seleccionada.

6.2 Informe escrito (Subida al aula virtual)

El informe debe ser subido al aula virtual en el plazo indicado por la cátedra. Debe incluir las siguientes secciones en el orden indicado:

1. Carátula con nombre del equipo, integrantes, módulo asignado y fecha.
2. Lista completa de requerimientos especificados, con todos los atributos solicitados.
3. Justificación de la herramienta de gestión seleccionada.
4. Capturas de pantalla de la herramienta, mostrando los requerimientos cargados y la simulación de estados.
5. Análisis de usabilidad: principios aplicados, diferencias Mobile/Desktop y decisiones de diseño justificadas.
6. Wireframes completos del flujo diseñado, con referencias explícitas a los requerimientos correspondientes.
7. Lecciones aprendidas del equipo.

ANEXO

Lecciones aprendidas

Se plantea al finalizar cada práctico el desarrollo de **Lecciones Aprendidas** por el equipo de trabajo.

- Las Lecciones Aprendidas son aspectos o experiencias observadas durante la realización de los trabajos prácticos que puedan ser tenidas en cuenta en trabajos prácticos futuros y los años siguientes, para las otras camadas que cursen la materia
- Las lecciones aprendidas permitirán mantener una gestión del conocimiento de los trabajos prácticos y de los contenidos de la materia, que facilitarán la repetición de éxitos y evitan la repetición de desaciertos en los desarrollos de las prácticas.
- La base de conocimiento almacenará las lecciones aprendidas y las soluciones aplicadas, asegurando la consistencia y control de la información la práctica de la materia, pudiendo ser reutilizada por los alumnos de diferentes años.

La experiencia adquirida pueden ser clasificadas de acuerdo a los intereses de los diferentes equipos de trabajo en la materia, tales como:

- La información de los proyectos/trabajos prácticos.
- Lo que ha fallado y lo que ha tenido éxito.
- Información de tecnología, herramientas y metodología.
- Información de instalación y uso de herramientas.
- Riesgo que han surgido en la implementación de las tecnologías, herramientas, metodología, lengua, plataforma y de los proyectos/trabajos prácticos.
- Información de las lecciones aprendidas en los equipos de trabajo.
- Información sobre defectos técnicos y como se han solucionado.

Principios de Usabilidad

Que es la usabilidad?

Es la facilidad con que las personas pueden utilizar una herramienta particular o cualquier otro objeto fabricado por humanos con el fin de alcanzar un objeto concreto. En interacción persona-servidor: Se refiere a la claridad y la elegancia con que se diseña la interacción con un programa de ordenador o sitio web.

Son técnicas que ayudan a los seres humanos a realizar tareas en entornos gráficos de un sistema.

Conceptos muy importantes.

Trabajamos para seres humanos, que quieren realizar una tarea de una forma sencilla y eficaz y en este caso particular, la deben realizar frente a un ordenador en un entorno gráfico, la web. La usabilidad ayuda a que esta tarea se realice de una forma sencilla analizando el comportamiento humano, y los pasos necesarios para ejecutar la tarea de una forma eficaz. Quizás suene un poco a trabajo industrial, pero al hacer referencia al término "tarea", estamos haciendo referencia a buscar un programa que queremos ver en la tele, a ver información sobre mi equipo de fútbol favorito, a comprar un libro sobre artes marciales.

Hace falta la usabilidad para hacer un sistema web?

Sea tu web un portal o una página personal, no olvides que la gente que entra en tu web es por que básicamente busca algo. Hacerlo sencillo es decisión tuya, pero seguro que entre todos podemos hacer de internet un sitio mejor donde la gente no se sienta perdida o extraviada o sencillamente defraudada.

Existen tres principios básicos de Usabilidad que nunca deben ser olvidados en el diseño de una interfaz:

Primer Principio: Reconocer la Diversidad

Es importantísimo siempre tener en mente que cada interfaz está hecha para diferentes tareas, no puede haber una interfaz que funcione efectivamente para todas las tareas. Además de que cada interfaz es debe estar orientada a una tarea en especial, también se debe tener en cuenta que no hay usuarios iguales.

Teniendo estos factores en cuenta existen una cantidad exorbitante de posibilidades para la creación de interfaces. La creación de interfaces para cada individuo es simplemente imposible, es por tanto, que los usuarios se deben categorizar. Las categorías pueden estar basadas en muchos criterios, por ejemplo el nivel de destreza o conocimientos de un

programa o tema (novicios, intermedios o expertos) cada uno con diferentes necesidades en las interfaces. Otro criterio puede ser el tipo de trabajo desempeñado, por ejemplo, una interfaz para una industria puede abarcar interfaces especiales para obreros, supervisor, gerente, etc. Es necesario analizar a fondo la forma en la cual se llevará a cabo la interacción entre el usuario y la interfaz. Se debe determinar el total de las tareas que pueden llevarse a cabo mediante la interfaz y cada una de estas separarse en acciones más pequeñas, y estas a su vez en otras aún más básicas.

Se debe tener un control del nivel de granularidad de las tareas más básicas, pues tener pocas causa problemas al tener que realizar muchas tareas básicas para realizar una tarea. Así como tener muy pocas tareas básicas lleva a crear muchas tener muchas tareas de alto nivel, confundiendo al usuario con el gran número de posibles acciones.

En el afán de crear la mejor interfaz posible, es necesario estudiar las formas en las cuales puede llevarse a cabo la interacción, teniendo en cuenta los requerimientos y posibilidades (físicas y mentales) de los usuarios. Así, podemos reconocer las siguientes formas de interacción:

- **Manipulación Directa.** Cuando el usuario interactúa con la representación visual del entorno sobre el que trabaja. Este tipo de interacción es sumamente sencilla para el usuario y extraordinariamente intuitiva. Requiere poco, o nulo, entrenamiento previo para el uso de la interfaz.
- **Selección de Menús.** Los usuarios seleccionan los elementos con los que se trabaja de una lista con posibilidades. Este tipo de interacción nos permite utilizar muy poca memoria para realizar los trabajos. Requiere mucho esfuerzo por parte de los diseñadores para asegurar que los elementos se encuentren donde tengan un significado para los usuarios.
- **Llenado de Formas.** Usado cuando es necesario ingresar datos a los sistemas. Es generalmente una forma en la cual los usuarios pueden introducir gran cantidad de errores. Es necesario ser claros en los tipos de datos pedidos, así como permitir a los usuarios muchas libertades como moverse a través de los cambios y que no todos los campos de la forma sean obligatorios.
- **Lenguajes de Comandos.** Los lenguajes de comandos son utilizados frecuentemente por usuarios de nivel avanzado. Los comandos dan gran flexibilidad y libertad de movimiento, así como dan la capacidad al usuario de mejorar las características del sistema. Son por lo general lentos de aprender, pero sumamente rápidos una vez dominados.
- **Lenguaje Natural.** La interacción mediante lenguaje natural entre una computadora y un humano es un tema que aún no ha sido resuelto. Es verdad que abre una gran cantidad de posibilidades, pues un usuario no tendría por qué tomar cursos de entrenamiento para utilizar el sistema (solo conocimiento previo de las tareas a realizar, lenguaje técnico, etc.). Hasta el momento, aunque existen muchos intentos de interfaces de lenguaje natural, el panorama actual no es muy alentador en dominios muy generales. Para dominios específicos de conocimiento y gramáticas reducidas, es posible encontrar buenos sistemas, aunque siempre faltando los elementos humanos por esenciales.

Segundo Principio: 8 reglas de Oro sobre la usabilidad

1. Esforzarse para conseguir consistencia.

Deberíamos exigirnos secuencias de acciones consistentes, utilizarse terminología consistente en los mensajes, menús y pantallas de ayuda, emplearse de forma consistente el color, composición, mayúsculas, fuentes, etc.

Las excepciones, como la necesaria confirmación de la orden de borrado o no mostrar los caracteres de las contraseñas, deberían ser comprensibles y limitadas en número.

- Todos los componentes de la interfaz deben ser uniformes y seguir un conjunto de reglas simples.
- Ayuda a generalizar conocimiento de un aspecto del sistema a los otros.
- Prever la frustración inducida cuando el sistema no se comporta de la manera esperada y lógica.

Ejemplos:

- Mismas palabras o códigos utilizados
- Posición y orden de controles y botones (ayuda, status, salida, etc.)
- Items de un menú colocados siempre en la misma posición (salir = 0)
- Comandos como Ayuda, Status, Copiar, etc. deben estar siempre disponibles
- Consistencia con el sistema operativo y otros programas

2. Atender la usabilidad universal.

Reconocer las necesidades de los diversos usuarios, facilitando la transformación del contenido. Diferencias entre segmentos: Principiante/experto, edades, discapacidades y diversidad tecnológica.

Ejemplo:

- Usuarios Casuales
- Tutoriales
- Wizards
- Prompts (qué hacer ahora)
- Ayuda
- En línea
- ToolTips
- Ventanas (sensibles a contexto)
- Modo Simple: Los comandos más complejos están escondidos
- Manipulación Directa
- Uso de Valores por Defecto
- Usuarios Expertos
- Shortcuts de teclado
- Línea de Comandos
- Modo Experto
- Eliminación de Prompts y Warnings
- Interfaz Extensible y Customizable

3. Ofrecer realimentación informativa

Para cada acción del sistema debería haber una realimentación por parte del sistema.

Aspectos:

- Qué está haciendo el sistema.
- Cómo se interpretan los comandos del Usuario.
- El usuario debe saber en todo momento que está pasando. Tipos de Feedback:
- Respuesta a un comando del usuario: Movimiento del cursor, selección de un Menú, etc.
- Estado Actual: Brocha seleccionada, color, posición, dirección de la carpeta, scroll bars.
- Procesamiento por parte de la máquina: instantáneos, cursores de espera, diálogos explicativos, barras de progreso.

4. Diseñar diálogos para conducir a la finalización

Crear sistemas claros de conducción hasta el final del proceso de las secuencias de acción del usuario.

Ej: Al final de una compra aparece una ventana o un texto que te indica que tú compra a finalizado.

- La interacción debe ser sencilla y comportarse de la manera en que el usuario lo espera.
- Minimizar elementos de la interfaz
- Menos para aprender, para equivocarse, distraerse.
- Orden Natural de la Información:
- Agrupar gráficamente información relacionada.
- El orden de acceso a la información debe ser como el usuario la espera
- Esconder o eliminar información irrelevante o usada ocasionalmente
- Puede distraer al usuario de las informaciones importantes.
- Utilizar el Lenguaje del Usuario:
- Usar terminologías e iconografías familiares para el Usuario
- Traducir mensajes de error al lenguaje del Usuario

5. Prevenir errores

Diseñar el sistema de forma que los usuarios no puedan cometer errores serios. Ej: deshabilitar elementos necesarios, etc...

Si fuera así el sistema debe de indicarle al usuario el error por medio de diferentes interfaces textuales o metafóricas.

- Nunca ofrecer un comando que lleve a un mensaje como: "Comando ilegal": Desactivarlo
- Borrar un objeto que no puede ser borrado
- Aplicar estilo de letra a un objeto que no sea texto
- Utilizar controles que impidan introducir datos erróneos:
- Campos de Texto numéricos
- Fechas
- Scroll Bars
- ListBoxes

-
- Los comandos que ofrece el sistema deben ser válidos en el contexto
 - Esto permite simplificar la interfaz, ofreciendo únicamente los comandos relevantes al contexto (ej: Diálogo de Selección, barra de Dibujos)
 - Advertir al Usuario
 - Es menos deseable, puesto que interrumpe el flujo de la interacción.

6. Permitir deshacer acciones de forma fácil

En la medida que podamos diseñar nuestro interfaz, el usuario puede tener la posibilidad de remediar acciones erróneas, por lo tanto construir un sistema reversible. Este sistema suaviza la ansiedad que se puede producir en el usuario. Ej: control + Z

Ejemplos: Deshacer (Undo): Para comandos completados, Abortar: Para comando en proceso y Cancelar: Para formas o conjunto de comandos.

7. Dar soporte al control interno

Los operadores experimentados desean firmemente tener la sensación de que están al mando de la interfaz y que la interfaz responde a sus acciones. Las acciones sorprendentes por parte de la interfaz, las secuencias de entrada de datos tediosas, la incapacidad o dificultad para obtener la información necesaria y la incapacidad para producir acciones deseadas, crea ansiedad e insatisfacción. EJ: panel de control de windows

8. Reducir la carga de la memoria a corto plazo

Diseñar sistemas de visualización simples, la frecuencia de movimiento de ventana se reduzca y que se asigne suficiente tiempo de entrenamiento para códigos y secuencias de acciones.

* La regla general que los humanos pueden recordar "siete más / menos dos elementos" de información)

Nota: Escribir bien para internet es todo un arte. Pero siguiendo las reglas básicas de: (1) poner las conclusiones al principio y (2) escribir como un 25% de lo que pondrías en un papel, se puede llegar muy lejos. Leer en pantalla cuesta mucho, por lo que, en el caso de textos para internet, reduce y simplifica todo lo que puedas.

Tercer Principio: Prevenir Errores

Es necesario partir del hecho que los usuarios son altamente propensos a cometer errores, por grandes o pequeños que los errores sean. Es necesario por tanto prevenir o ayudar a corregir los errores mediante el uso de mensajes altamente descriptivos sobre el origen de los errores y posibilidades de corrección. Una manera muy simple de prevenir en editores es proveer de comandos de muy alto nivel que realicen mediante una sola orden acciones completas. Otra manera de corregir o prevenir errores es corregir "en línea" los posibles errores, como por ejemplo en lenguajes de comandos antes de ejecutar una orden el intérprete de órdenes puede verificar la existencia de los comandos y que los argumentos sean correctos.

Links Recomendados:

- <http://materias.fi.uba.ar/7507/content/20101/lecturas/usabilidad.pdf>
- <https://webdesign.tutsplus.com/articles/8-golden-rules-for-better-interface-design--cms-30886>
- <http://theomandel.com/wp-content/uploads/2012/07/Mandel-GoldenRules.pdf>
- <https://public-media.interaction-design.org/pdf/Shneiderman.s.Eight.Golden.Rules.Worksheet.pdf>
- <https://www.interaction-design.org/literature/article/shneiderman-s-eight-golden-rule-s- will-help-you-design-better-interfaces>
- http://cs.gettysburg.edu/~skim/cs391/lectures/20_6pp.pdf
- <https://www.youtube.com/watch?v=H-doP3il6VY>

Evolución del Diseño

Las primeras escuelas de diseño (Bauhaus o Ulm) lo enseñaban de manera centralizada, como un único Diseño. Comprendía la aplicación de criterios y metodologías a todas las ramas que se conocían.

A medida que fueron creciendo y complejizándose las industrias, fueron demandando conocimientos específicos en distintos campos y esto generó que de a poco se fueran separando las especialidades hasta llegar, más de medio siglo después, a dividir el estudio del diseño en gráfica, de productos, de indumentaria, entre otras.

En la actualidad vivimos en una sociedad informatizada e interconectada mediante diversos dispositivos con los cuales las personas interactúan.

El espacio físico de interacción humano-computadora contenido en la pantalla ha creado un diálogo entre usuario y diseño nunca antes experimentado que requiere de conocimientos específicos para lograr experiencias enriquecedoras.

La cantidad y calidad de dichas interacciones han creado una nueva dimensión del diseño que se diferencia de las demás ramas a tal punto que está creando un campo autónomo e independiente de diseño. Esto es lo que se conoce como Diseñador UX.

User Experience - UX

El termino User Experience fue dado a conocer por Donald Norman en 1995 en un artículo titulado: "User Experience Architect, in the mid-1990's"

"El resultado de las percepciones y respuestas de una persona por el uso y uso anticipado de un producto, sistema o servicio. "

"Inventé el término porque pensé que el diseño de interfaces humanas y la usabilidad abarcaban demasiado poco: yo quería cubrir todos los aspectos de la experiencia de las personas con un sistema, incluido el diseño industrial, gráfico, de interfaces, la interacción física y la manual"

UX Design (User Experience Design) o "Diseño de Experiencia de Usuario" es una filosofía de diseño que tiene por objetivo la creación de productos que resuelvan necesidades concretas de sus usuarios, consiguiendo la mayor satisfacción y mejor experiencia de uso posible con el mínimo esfuerzo.

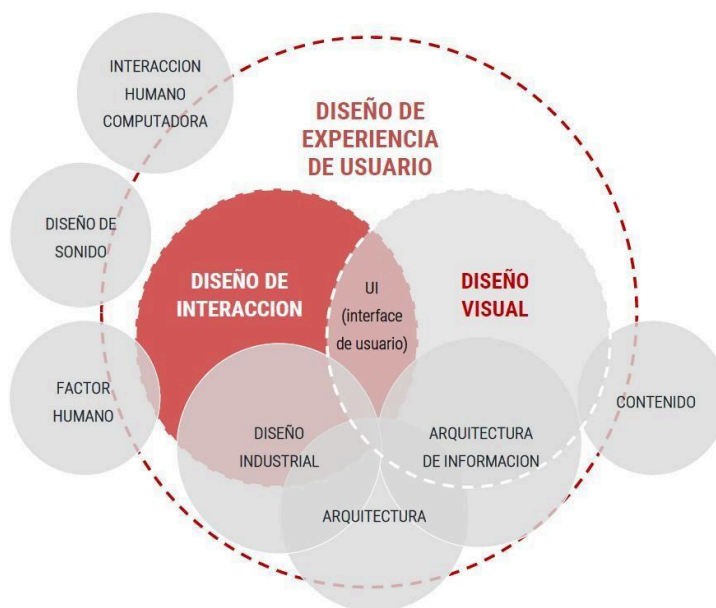
- Es un proceso en el que se utilizan una serie de técnicas multidisciplinarias y donde cada decisión debe estar basada en las necesidades, objetivos, expectativas, motivaciones y capacidades de los usuarios.

Framework o esqueleto de UX

La mayoría de los procesos que hacen Diseño Centrado en el Usuario tienen el siguiente esqueleto:

1. Conocer a fondo a los usuarios finales, normalmente a través de una intensa investigación cualitativa o cuantitativa.
2. Diseñar un producto que resuelva sus necesidades y se ajuste a sus capacidades, expectativas y motivaciones.
3. Poner a prueba lo diseñado, aplicando constantes test de usuario durante el proceso de diseño.

Elementos de UX



Componentes que forman a un UX

Diseño de interacción
 Arquitectura de la información
 Creación de perfiles de usuario
 Creación de wireframes
 Creación de documentos de diseño de interacción
 Diseño de interfaces para dispositivos móviles y otros
 Contrucción de interacciones digitales
 Diseño y rediseño de SAAS
 Diseño Web

Perfil y responsabilidades de un UX

Saber identificar y analizar las necesidades de la sociedad digital.
 Dominar el proceso de comunicación y los códigos del lenguaje digital multimedia.
 Entender los requerimientos de un proyecto, estructurarlos para definir la arquitectura de los contenidos y el diseño de experiencia del usuario.
 Interpretar la jerarquía de la información y la usabilidad de las aplicaciones en función del entorno.
 Diseñar interfaces altamente intuitivas, desde la planificación, la elección de la iconografía y el concepto estético implícito.
 Entender los fundamentos de la interacción en cada tipo de interfaz y diseñar interfaces adaptadas a los diferentes dispositivos y usuarios.
 Practicar el diseño colaborativo mediante equipos multidisciplinares en el contexto internacional, multicultural.

Conocimientos y habilidades técnicas requeridas en un buen perfil UX

Maquetación (HTML, XHTML, XML)
 Código CSS
 Diseño de página, diseño de interface
 Programación front-end (JavaScript Angular)
 Edición y producción de imágenes
 Arquitectura de información, wireframes, site- maps
 Testeo / conocimiento de usabilidad
 Diseño Gráfico (Diseño Visual)
 Desarrollo back-end (PHP, Ruby on Rails, ASP)
 Administración y gestión de Proyectos
 Conocimientos de metodologías como Desing Thinking (ver abajo)
 Desarrollo de contenido y edición
 Optimización y marketing en motores de búsqueda
 Testeo interfaz usuaria / conocimiento de accesibilidad

Design Thinking es tener un pensamiento de diseño para analizar algo (un problema), tal y como lo haría un diseñador, evidentemente con la finalidad de encontrar una solución. Para ello se utilizan las herramientas, componentes y aspectos creativos inherentes a la profesión de diseño y a la persona que realiza la acción de diseñar (el diseñador, el thinker).

Design Thinking es una manera de ofrecer una solución a un problema. Descomponemos un problema, lo dividimos en partes más pequeñas, las analizamos, pensamos mucho, sin límites, todo lo que podamos y todo lo que se nos ocurra, de manera empática y junto a otros miembros del equipo, entonces estaremos mucho más cerca de encontrar la solución que buscamos.

Una característica fundamental de la metodología Design Thinking es que está centrada en el usuario y en los problemas que a éste se le pueden plantear y en la empatía. Estos conceptos están estrechamente relacionados, es primordial hacer previamente una composición de lugar, un análisis de la situación, tener conciencia de donde estamos y qué necesitamos.

Primero identificamos el problema que tenemos que resolver, incluso plantearemos nosotros nuevos problemas, nuevos interrogantes, nuevos cuestionamientos para contextualizar mucho mejor la situación, ser conscientes del punto de partida en todas las vertientes posibles, en 360 grados. Tenemos que ser curiosos y cuestionarnos todo con un gran deseo de mejorar y de aprender, cualidad primordial del Thinker. Que no se nos escape nada y tenemos que darnos cuenta de todo, de cosas que a simple vista o de manera rápida no pudiéramos apreciar o valorar. Tenemos que ser empáticos, por medio de la empatía nos integramos en el entorno y tratamos de adaptarnos a él. Nos identificamos con el usuario y su problema, fundamental para poder ayudarlo a resolverlo. El ambiente y el usuario nos condicionan y hay que conocerlos, entenderlos, fusionarnos y relacionarnos con él.

Links Recomendados:

Ver: <https://www.luisan.net/blog/disenio-grafico/que-es-design-thinking>